

System Engineer: Take-Home Assignment

Overview

Design and implement a real-time data channel between three pieces of software that run on: a **Robot**, a **Cloud Service**, and a **User PC**. When a robot is turned on, the system should dynamically spawn a cloud-side component and establish a peer-to-peer network mesh where all three parties can publish and subscribe to messages to each other directly.

Problem Statement

You are building a system where:

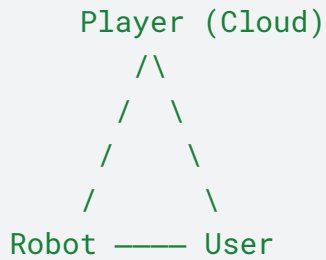
- **Robot** - A piece of software that runs on edge device (simulated) that connects to the cloud when it starts up, and also controls the robot via robot's python sdk (for example, a fake robot with the jetbot style <https://github.com/NVIDIA-AI-IOT/jetbot/tree/master/jetbot>)
- **User** - A client application (CLI script) that connects to the cloud service to communicate with a specific robot. Think of this as a remote operator's terminal or control dashboard.
- **Cloud Service** - A backend server that orchestrates connections and spawns a **Player** (a cloud-side component/process) for each robot that registers and connects to

The Goal

When all three entities connect, they should form a **triangle data channel** where:

- Robot can send/receive data to/from User and Player, and apply those commands to robot
- User can send/receive data to/from Robot and Player
- Player can send/receive data to/from Robot and User

None



Requirements

Functional Requirements

- When a robot starts, it pings the cloud server and starts sending heartbeat messages
- The cloud service should maintain connection and status of this robot
- User connects to cloud server and can
 - List connected robots
 - Connect to one of the robot
- When user requests to connect to a robot, cloud server spawns up a pub/sub server, after success, user and robot are also directed to spawn up pub/sub server respectively with information about the pub/sub server on the cloud side
- Three pub/sub servers should connect and form a triangle network mesh
 - Each entity should start to publish messages and subscribe to each other's publishing channel directly

Implementation

Use any pub/sub server with the following topics

Topic	Publisher	Subscriber(s)	Purpose
<code>robot/{robot_id}/sensor</code>	Robot	Player, User	Robot sends raw sensor data
<code>robot/{robot_id}/command</code>	User	Robot	User sends commands to robot, robot acts with those commands accordingly
<code>robot/{robot_id}/processed</code>	Player	User	Player sends processed/analyzed data

<code>robot/{robot_id}/status</code>	Any	All	Status updates from any entity
--------------------------------------	-----	-----	--------------------------------

Note: Multiple entities can subscribe to the same topic. When a publisher sends data to a topic, all subscribers who subscribe to it should receive it.

Demonstration

Simulate the flow in the functional requirements in single computer (or multiple computers if conditions permit), and show the triangle communication working:

- **Robot** publishes sensor data every second: `{ "state": 25.5 }`
- **Player** receives sensor data, processes, and republishes: `{ "state": 25.5, "status": "normal" }`
- **User** publishes data, player receives receives data, user publishes a command: `{ "command": "stop" }, { "command": "forward" }`
- **Robot** receives the command and executes the command (by calling robot's sdk or other approaches, use jetbot style api as an example)

Each entity should print all data it receives from topics it's subscribed to.

Non-Functional Requirements

- Clean, readable code with clear separation of concerns
 - Handle edge cases and connection failure in networks (robot not found, duplicate registration, player spawn failure, etc.)
 - Proper resource management
 - Testing and benchmarking
-

Technical Constraints

- **Language:** any programming language of your choice
-

Deliverables

Source Code (GitHub repo)

None

```
repo/  
├─ cloud_service/      # cloud server  
├─ robot/              # Robot simulator  
├─ user/               # User client  
├─ README.md  
└─ requirements.txt    # dependencies
```

Evaluation Criteria

Criteria	Weight	What We're Looking For
Correctness	40%	Does it work as specified? All three entities communicate?
Architecture	30%	Clean design, separation of concerns, proper abstractions
Documentation	20%	Clear README with all the 5 points mentioned above.
Code Quality	10%	Readable, maintainable, error handling, no hardcoded values
Bonus	10%	Security, scalability, error handling, careful resource management, performance

Bonus

1. **Security:** protect communication
 2. **Scalability:** how to support multiple robots for multiple users
 3. **Error handling and resource management**
 4. **Benchmark performance:** latency
 5. **Testing**
-

If anything is ambiguous, please feel free to make assumptions, just clearly state in the code what assumptions are being made.